OAVS EXAM SERIES

Odisha Adarsha Vidyalaya Sangathan

Computer Teacher (TGT)

EXAM PREPARATION SERIES

ANSI C PROGRAMMING

Prompt 2

Identifiers and Keywords

in ANSI C

120+ MCQs | **Answer Key** | **Explanations** | **Mock Test**

TOPICS COVERED

Rules for Identifiers	Valid & Invalid Identifiers	ANSI C Keywords
Naming Conventions	Reserved Words	Code-Based Questions

120+	5	30	100%
MCQs	Sections	Mock Test Questions	Coverage of Syllabus

Prepared for: OAVS Computer Teacher (TGT) Examination | ANSI C (C89/C90 Standard) | 2025 Edition

TABLE OF CONTENTS

Quick Reference: ANSI C89 Keywords	All 32 Keywords at a Glance	**Pg 4**
Section 1: Rules for Identifiers	Q1 – Q25 | 25 Questions	**Pg 5**
Section 2: Valid and Invalid Identifiers	Q26 – Q55 | 30 Questions	**Pg 7**
Section 3: ANSI C Keywords	Q56 – Q85 | 30 Questions	**Pg 9**
Section 4: Naming Conventions	Q86 – Q100 | 15 Questions	**Pg 11**
Section 5: Reserved Words	Q101 – Q120 | 20 Questions	**Pg 12**
Mock Test	30 Questions — Timed Practice	**Pg 14**
Complete Answer Key	All Sections + Mock Test	**Pg 16**

QUICK REFERENCE

All 32 ANSI C89 Keywords

Data Types	Control Flow	Storage Class	Type Qualifiers	Misc / Operators
char	if	auto	const	sizeof
short	else	register	volatile	typedef
int	for	static		struct
long	while	extern		union
float	do			enum
double	switch			return
void	case			goto
signed	break			
unsigned	continue			
	default			
	goto			
	return			

NOTE: The 32 ANSI C89 keywords are: auto, break, case, char, const, continue, default, do, double, else, enum, extern, float, for, goto, if, int, long, register, return, short, signed, sizeof, static, struct, switch, typedef, union, unsigned, void, volatile, while.

Section 1: Rules for Identifiers

Q1. Rules for Identifiers

Which of the following is the correct rule for forming an identifier in ANSI C?

(A) An identifier can begin with a digit

(B) An identifier can contain special characters like @

(C) ✓ An identifier must begin with a letter or underscore

(D) An identifier can have spaces

Explanation: ANSI C mandates that an identifier starts with a letter (a-z, A-Z) or an underscore (_), followed by letters, digits, or underscores.

Q2. Rules for Identifiers

What is the maximum length of an identifier that is guaranteed to be significant in ANSI C (C89)?

(A) 8 characters

(B) ✓ 31 characters

(C) 64 characters

(D) Unlimited

Explanation: ANSI C (C89) guarantees significance for the first 31 characters of an identifier. Characters beyond 31 may be ignored by the compiler.

Q3. Rules for Identifiers

Which character is allowed as the first character of an identifier in C?

(A) Digit (0-9)

(B) Dollar sign ($)

(C) ✓ Underscore (_)

(D) Hyphen (-)

Explanation: Only letters (a-z, A-Z) and the underscore (_) are valid as the first character of a C identifier. Digits and special characters are not allowed at the start.

Q4. Rules for Identifiers

Are C identifiers case-sensitive?

(A) No, C is not case-sensitive

(B) ✓ Yes, 'Value' and 'value' are different identifiers

(C) Only keywords are case-sensitive

(D) Only function names are case-sensitive

Explanation: C is a case-sensitive language. 'Value', 'value', and 'VALUE' are three distinct identifiers.

Q5. Rules for Identifiers

Which of the following characters CANNOT appear in a C identifier?

(A) Underscore (_)

(B) Uppercase letter

(C) Digit

(D) ✓ Hyphen (-)

Explanation: Hyphens (-) are not allowed in C identifiers. Valid characters are: letters (a-z, A-Z), digits (0-9), and underscore (_).

Q6. Rules for Identifiers

An identifier in C can be used to name which of the following?

(A) Variables only

(B) Functions only

(C) ✓ Variables, functions, arrays, and labels

(D) Only user-defined types

Explanation: Identifiers in C are used to name variables, functions, arrays, structures, labels, and other user-defined items.

Q7. Rules for Identifiers

Which statement about ANSI C identifiers is FALSE?

(A) Identifiers are case-sensitive

(B) ✓ Identifiers can start with digits

(C) Identifiers can contain underscores

(D) Identifiers cannot be keywords

Explanation: Identifiers cannot start with a digit in ANSI C. They must begin with a letter or underscore.

Q8. Rules for Identifiers

How many characters of an external identifier are guaranteed to be significant in ANSI C (C89)?

(A) ✓ 6 characters

(B) 8 characters

(C) 31 characters

(D) 64 characters

Explanation: For external identifiers (linked across files), ANSI C89 guarantees only the first 6 characters are significant, due to linker limitations on some systems.

Q9. Rules for Identifiers

Which rule correctly describes ANSI C identifier naming?

(A) Spaces are allowed if enclosed in quotes

(B) Only lowercase letters are allowed

(C) ✓ Can contain letters, digits and underscores but must not start with a digit

(D) Must be at least 3 characters long

Explanation: An identifier consists of letters, digits, and underscores, but must not begin with a digit.

Q10. Rules for Identifiers

Which of the following is NOT a valid token type for identifiers?

(A) Letter followed by letters

(B) Underscore followed by digits

(C) ✓ Digit followed by letters

(D) Letter followed by underscore

Explanation: A digit cannot begin an identifier. 'Digit followed by letters' like '3abc' is invalid.

Q11. Rules for Identifiers

In ANSI C, identifiers beginning with double underscore (__) are:

(A) Encouraged for user variables

(B) ✓ Reserved for compiler/system use

(C) Always invalid

(D) Used only for macros

Explanation: Names beginning with __ (double underscore) or _ followed by uppercase are reserved for the implementation (compiler and standard library). User programs should avoid such names.

Q12. Rules for Identifiers

Which of the following is TRUE about C identifiers?

(A) An identifier cannot start with an underscore

(B) ✓ All identifiers must be declared before use

(C) Identifiers are limited to 8 characters

(D) Only ASCII characters are used in identifiers

Explanation: In C, all identifiers must be declared before they are used (with the exception of function names under implicit int rule in C89, which is still a declaration).

Q13. Rules for Identifiers

What happens if two identifiers differ only after the 31st character in ANSI C89?

(A) Compilation error occurs

(B) ✓ They may be treated as identical by the compiler

(C) Only the first one is used

(D) A warning is issued

Explanation: ANSI C89 guarantees only the first 31 characters of internal identifiers are significant. Characters beyond position 31 may be ignored.

Q14. Rules for Identifiers

Which of these is a valid C identifier according to ANSI C rules?

(A) 2fast

(B) ✓ fast_2

(C) fast-2

(D) fast 2

Explanation: 'fast_2' is valid — starts with a letter, contains letters, digit, and underscore. '2fast' starts with a digit; 'fast-2' has a hyphen; 'fast 2' has a space.

Q15. Rules for Identifiers

The identifier '_count' (single underscore followed by lowercase) in a user program is:

(A) Always illegal

(B) A keyword in C

(C) ✓ Valid but potentially conflicts with library names

(D) Reserved by ANSI C and always illegal

Explanation: Identifiers starting with a single underscore followed by a lowercase letter are technically valid but discouraged as they may conflict with POSIX or library-defined names.

Q16. Rules for Identifiers

Which of the following is the best practice for naming identifiers in C?

(A) Start all identifiers with a digit for uniqueness

(B) ✓ Use meaningful names that reflect purpose

(C) Use only single-character identifiers

(D) Always use capital letters only

Explanation: Best practice in C (and all programming) is to use meaningful, descriptive names. This improves code readability and maintainability.

Q17. Rules for Identifiers

Consider: 'int _VAR = 5;' in a user program. Which is correct?

(A) Completely illegal in C

(B) Always causes runtime error

(C) ✓ Reserved for system use and should be avoided by programmers

(D) Perfectly valid and recommended

Explanation: Identifiers beginning with underscore followed by an uppercase letter (like _VAR) are reserved for the implementation. User programs should not define such identifiers.

Q18. Rules for Identifiers

Which character set is used for ANSI C identifiers?

(A) Only lowercase letters a-z

(B) ✓ Letters a-z, A-Z, digits 0-9, and underscore

(C) All printable ASCII characters

(D) Unicode characters

Explanation: ANSI C identifier characters are restricted to: letters (a-z, A-Z), digits (0-9), and underscore (_). No other character is allowed.

Q19. Rules for Identifiers

Can the same name be used for both a variable and a function in the same scope in C?

(A) Yes, C allows it freely

(B) ✓ No, each identifier must be unique in its scope

(C) Only if they have different parameter counts

(D) Yes, because functions and variables are in different namespaces

Explanation: Within the same scope, an identifier can only refer to one entity. Using the same name for a variable and function in the same scope causes a compilation error.

Q20. Rules for Identifiers

Which of the following is the MOST restrictive rule for C identifiers?

(A) Must end with a letter

(B) ✓ Cannot begin with a digit

(C) Must contain at least one digit

(D) Cannot exceed 10 characters

Explanation: The most fundamental and restrictive rule is that identifiers cannot begin with a digit. They must start with a letter or underscore.

Q21. Rules for Identifiers

In ANSI C, are uppercase letters allowed in identifiers?

(A) No, only lowercase is allowed

(B) ✓ Yes, uppercase letters are allowed

(C) Only in macro names

(D) Only as the first character

Explanation: Uppercase letters are fully allowed anywhere in a C identifier (except as the first character restriction that applies to digits, not letters).

Q22. Rules for Identifiers

What is the result of using a keyword as an identifier in C?

(A) A runtime error

(B) ✓ A compilation error

(C) A warning only

(D) Undefined behavior

Explanation: Keywords are reserved words in C. Using a keyword like 'int', 'for', or 'while' as an identifier name will result in a compilation (syntax) error.

Q23. Rules for Identifiers

Which of the following identifiers would cause a conflict with standard C library names?

(A) myprintf

(B) ✓ printf

(C) my_print

(D) PRINTF

Explanation: 'printf' is a standard C library function. Redefining it as a user identifier, while technically possible in limited contexts, causes conflicts and is strongly discouraged.

Q24. Rules for Identifiers

How does C treat the identifiers 'TOTAL' and 'total'?

(A) They are identical

(B) ✓ They are different identifiers

(C) 'TOTAL' is a keyword

(D) Only one can be used in a program

Explanation: C is case-sensitive. 'TOTAL' and 'total' are completely different identifiers and can coexist in the same scope.

Q25. Rules for Identifiers

A valid C identifier must:

(A) Contain at least one uppercase letter

(B) ✓ Not match any keyword and start with letter or underscore

(C) Be shorter than 8 characters

(D) Contain at least one digit

Explanation: A valid identifier must not be a keyword, must start with a letter or underscore, and can contain only letters, digits, and underscores.

Section 2: Valid and Invalid Identifiers

Q26. Valid and Invalid Identifiers

Which of the following is a VALID C identifier?

(A) 123abc

(B) ✓ _myVar

(C) my-var

(D) my var

Explanation: '_myVar' is valid: starts with underscore, followed by letters. '123abc' starts with digit; 'my-var' has hyphen; 'my var' has space.

Q27. Valid and Invalid Identifiers

Which identifier is INVALID in ANSI C?

(A) _123

(B) abc123

(C) ABC

(D) ✓ int

Explanation: 'int' is a keyword in C and therefore cannot be used as an identifier. The other three are syntactically valid identifiers.

Q28. Valid and Invalid Identifiers

Which of the following is a valid C identifier?

(A) $variable

(B) variable$

(C) ✓ _variable_

(D) #variable

Explanation: '_variable_' starts with underscore and contains only letters and underscores — valid. $ and # are not allowed in C identifiers.

Q29. Valid and Invalid Identifiers

Identify the INVALID identifier: rollno, roll_no, rollno1, 1rollno

(A) rollno

(B) roll_no

(C) rollno1

(D) ✓ 1rollno

Explanation: '1rollno' begins with a digit which is not allowed. All others are valid: 'rollno' (letters only), 'roll_no' (underscore allowed), 'rollno1' (digit at end allowed).

Q30. Valid and Invalid Identifiers

Which of the following identifiers is valid?

(A) while

(B) _while

(C) ✓ While

(D) WHILE_LOOP!

Explanation: 'While' (capital W) is a valid identifier — not a keyword. 'while' is a keyword; '_while' is valid but reserved by convention; 'WHILE_LOOP!' has exclamation mark which is invalid.

Q31. Valid and Invalid Identifiers

Which of these is NOT a valid C identifier?

(A) _

(B) a

(C) Z99

(D) ✓ 99Z

Explanation: '99Z' begins with digits — invalid. '_' (single underscore), 'a' (single letter), and 'Z99' (letter then digit) are all valid identifiers.

Q32. Valid and Invalid Identifiers

Select the valid identifier from: float, _float, Float, flo@t

(A) float

(B) _float

(C) ✓ Float

(D) flo@t

Explanation: 'Float' (capital F) is a valid identifier. 'float' is a keyword; 'flo@t' has @ which is invalid; '_float' is valid but reserved by convention.

Q33. Valid and Invalid Identifiers

Which of the following CANNOT be a valid identifier?

(A) _1a

(B) A1_

(C) a_1

(D) ✓ 1_a

Explanation: '1_a' begins with a digit — invalid. '_1a' starts with underscore, 'A1_' starts with letter, 'a_1' starts with letter — all valid.

Q34. Valid and Invalid Identifiers

Is 'goto' a valid user-defined identifier in C?

(A) Yes, it is valid

(B) ✓ No, it is a keyword

(C) Only in C99

(D) Valid if written as 'Goto'

Explanation: 'goto' is one of the 32 keywords in ANSI C (C89). It cannot be used as a user-defined identifier. However, 'Goto' (capital G) would be valid.

Q35. Valid and Invalid Identifiers

Which identifier is valid in C?

(A) my.var

(B) my:var

(C) ✓ my_var

(D) my+var

Explanation: Only underscore (_) is a valid non-alphanumeric character in identifiers. Dots, colons, and plus signs are not allowed.

Q36. Valid and Invalid Identifiers

Choose the valid identifier: return, Return, RETURN_, _return

(A) return

(B) Return

(C) RETURN_

(D) ✓ Both B and C

Explanation: 'return' is a keyword (invalid as identifier). 'Return' (capital R) and 'RETURN_' are both valid identifiers. '_return' is valid syntactically but reserved by convention.

Q37. Valid and Invalid Identifiers

Which of the following is a valid C identifier according to ANSI C rules?

(A) break

(B) Break

(C) BREAK

(D) ✓ Both B and C

Explanation: 'break' is a C keyword. 'Break' and 'BREAK' differ in case from 'break', making them valid user identifiers (though using them is poor practice).

Q38. Valid and Invalid Identifiers

Examine: sum, Sum, SUM, _sum, sum_. How many of these are valid identifiers?

(A) 2

(B) 3

(C) 4

(D) ✓ 5

Explanation: All five are valid identifiers. 'sum' is not a keyword (unlike 'int' or 'for'). All start with a letter or underscore and contain only alphanumerics and underscore.

Q39. Valid and Invalid Identifiers

Which of the following identifiers is invalid?

(A) student_name

(B) studentName

(C) ✓ student name

(D) StudentName

Explanation: 'student name' contains a space — spaces are not allowed in identifiers. The others use underscore or camelCase which are both valid.

Q40. Valid and Invalid Identifiers

Is '__FILE__' a valid user-defined identifier?

(A) Yes, completely valid

(B) ✓ No, it is a predefined macro

(C) Only in C99 standard

(D) Valid but generates warning

Explanation: '__FILE__' is a predefined macro in C that expands to the current filename. Names beginning and ending with double underscores are reserved by the implementation.

Q41. Valid and Invalid Identifiers

Which of the following is a valid identifier?

(A) 3_D_array

(B) ✓ array_3D

(C) array 3D

(D) array-3D

Explanation: 'array_3D' starts with a letter and contains only letters, digits, and underscores — valid. '3_D_array' starts with digit; 'array 3D' has space; 'array-3D' has hyphen.

Q42. Valid and Invalid Identifiers

Which identifier would be flagged as using a reserved naming convention?

(A) myVar

(B) ✓ __myVar

(C) MyVar

(D) my_Var

Explanation: '__myVar' starts with double underscore — a naming pattern reserved for the implementation (compiler and standard library).

Q43. Valid and Invalid Identifiers

Identify the valid identifier from this list: for, FOR, _for, 4for

(A) for

(B) ✓ FOR

(C) _for

(D) 4for

Explanation: 'for' is a keyword. '4for' starts with digit. '_for' starts with underscore — valid but reserved by convention. 'FOR' is fully valid as a user identifier (all caps, not a keyword).

Q44. Valid and Invalid Identifiers

Which of these identifiers is valid in ANSI C?

(A) long

(B) Long

(C) LONG

(D) ✓ Both B and C

Explanation: 'long' is a C keyword. Both 'Long' and 'LONG' differ in case and are therefore valid user-defined identifiers.

Q45. Valid and Invalid Identifiers

Is the identifier 'main' a keyword in C?

(A) Yes, it is a reserved keyword

(B) ✓ No, it is a user-defined identifier with special purpose

(C) It is a standard library function

(D) It varies by compiler

Explanation: 'main' is NOT a keyword in C. It is a user-defined identifier with special meaning (the program entry point). You could technically use 'main' as a variable name in some contexts (poor practice).

Q46. Valid and Invalid Identifiers

Which of the following identifiers is invalid due to using a special character?

(A) _x_

(B) x_1

(C) ✓ x@1

(D) X1

Explanation: 'x@1' contains the '@' symbol which is not allowed in C identifiers. Only letters, digits, and underscores are permitted.

Q47. Valid and Invalid Identifiers

Which statement is correct about the identifier 'NULL' in C?

(A) NULL is a keyword

(B) ✓ NULL is a predefined macro (not a keyword)

(C) NULL is an invalid identifier

(D) NULL is a data type

Explanation: 'NULL' is a predefined macro defined in that typically expands to 0 or (void)0. It is NOT a keyword, but redefining it as a user identifier is extremely poor practice.*

Q48. Valid and Invalid Identifiers

Which of the following uses of an identifier is correct?

(A) int 1stValue = 10;

(B) ✓ int first_value = 10;

(C) int first-value = 10;

(D) int first value = 10;

Explanation: 'first_value' is a valid identifier. '1stValue' starts with digit; 'first-value' has hyphen (treated as minus operator); 'first value' has a space.

Q49. Valid and Invalid Identifiers

The identifier 'EOF' in C is:

(A) A keyword

(B) ✓ A predefined macro constant

(C) An invalid identifier

(D) A data type

Explanation: 'EOF' is a predefined macro (typically -1) defined in to represent end-of-file. It is not a keyword but should not be redefined by users.

Q50. Valid and Invalid Identifiers

Which of the following is NOT a valid reason for an identifier to be invalid?

(A) Starts with a digit

(B) Contains a hyphen

(C) ✓ Contains 25 characters

(D) Matches a keyword

Explanation: An identifier containing 25 characters is valid — it is well within the 31-character significance guarantee. The other three are actual reasons for invalidity.

Q51. Valid and Invalid Identifiers

What is wrong with the identifier 'my#var' in C?

(A) Too long

(B) ✓ # is not a valid identifier character

(C) Starts with lowercase

(D) Nothing is wrong

Explanation: The '#' character is the preprocessor directive indicator in C and is not a valid character within identifiers. Only letters, digits, and underscores are allowed.

Q52. Valid and Invalid Identifiers

Which of the following sets contains ONLY valid C identifiers?

(A) sum, 2sum, _sum

(B) ✓ _sum, sum2, Sum

(C) int, Sum, _sum

(D) sum-2, sum_2, sum2

Explanation: '_sum' (starts with _), 'sum2' (letter then digit), 'Sum' (starts with uppercase) — all valid. 'int' is a keyword; '2sum' starts with digit; 'sum-2' has hyphen.

Q53. Valid and Invalid Identifiers

Consider the code: int _Val; Is this valid in ANSI C?

(A) Invalid — underscore cannot start an identifier

(B) ✓ Valid syntactically but '_Val' is reserved for implementation

(C) Valid and recommended

(D) Invalid — cannot mix case

Explanation: '_Val' starts with underscore followed by uppercase V, which is reserved for implementation use. It is syntactically valid but using such names in user code invokes undefined behavior per the C standard.

Q54. Valid and Invalid Identifiers

Which of the following is a valid identifier in ANSI C?

(A) case

(B) CASE

(C) Case

(D) ✓ Both B and C

Explanation: 'case' is a keyword in C. 'CASE' and 'Case' differ in case from the keyword and are both valid user-defined identifiers (though confusing and poor practice).

Q55. Valid and Invalid Identifiers

An identifier beginning with underscore and followed by an uppercase letter in C is:

(A) Completely safe to use

(B) ✓ Reserved and should be avoided by user programs

(C) Only valid in C99 and later

(D) Treated as a macro

Explanation: Per the C standard, identifiers beginning with underscore followed by uppercase letter (e.g., _A, _MAX) are reserved for the implementation. User programs must avoid defining such identifiers.

Section 3: ANSI C Keywords

Q56. ANSI C Keywords

How many keywords are defined in ANSI C (C89/C90)?

(A) 27

(B) 30

(C) ✓ 32

(D) 35

Explanation: ANSI C89/C90 defines exactly 32 keywords. C99 added 5 more (inline, restrict, _Bool, _Complex, _Imaginary) and C11 added more. But the base ANSI C count is 32.

Q57. ANSI C Keywords

Which of the following is NOT a keyword in ANSI C?

(A) auto

(B) register

(C) ✓ string

(D) extern

Explanation: 'string' is not a keyword in C (it is in C++, Java, and others). In C, strings are arrays of char. 'auto', 'register', and 'extern' are all valid C keywords.

Q58. ANSI C Keywords

Which keyword is used to declare a variable whose value cannot be changed?

(A) static

(B) ✓ const

(C) volatile

(D) register

Explanation: The 'const' keyword declares a variable as read-only (its value cannot be modified after initialization). Attempting to modify it causes a compilation error.

Q59. ANSI C Keywords

What does the 'extern' keyword indicate in C?

(A) The variable is external to all functions

(B) ✓ The variable is defined in another file/translation unit

(C) The variable is stored in external memory

(D) The function is external to the main function

Explanation: 'extern' declares that a variable or function is defined in another translation unit (source file). It provides a declaration without a definition, enabling cross-file variable sharing.

Q60. ANSI C Keywords

Which keyword is used to define a symbolic name for a data type?

(A) define

(B) ✓ typedef

(C) struct

(D) union

Explanation: 'typedef' creates an alias (alternate name) for a data type. Example: typedef unsigned int uint; allows 'uint' to be used as an alias for 'unsigned int'.

Q61. ANSI C Keywords

The keyword 'volatile' tells the compiler that:

(A) The variable cannot be modified

(B) ✓ The variable value may change unexpectedly (e.g., by hardware)

(C) The variable is stored in CPU registers

(D) The variable is shared between functions

Explanation: 'volatile' prevents the compiler from optimizing accesses to a variable, because its value may be changed by external factors (hardware, interrupt handlers, another thread).

Q62. ANSI C Keywords

Which ANSI C keyword is used for jump to a label?

(A) break

(B) continue

(C) ✓ goto

(D) return

Explanation: 'goto' unconditionally transfers program control to a labeled statement. While valid in C, its use is generally discouraged as it makes code harder to follow.

Q63. ANSI C Keywords

What is the purpose of the 'register' keyword in C?

(A) Declares a global variable

(B) ✓ Suggests the compiler store the variable in a CPU register

(C) Registers a function with the OS

(D) Reserves memory at compile time

Explanation: 'register' is a hint to the compiler to store the variable in a CPU register for faster access. Modern compilers largely ignore this hint as they optimize register allocation automatically.

Q64. ANSI C Keywords

Which of the following keywords is used to return a value from a function?

(A) exit

(B) ✓ return

(C) break

(D) yield

Explanation: 'return' transfers control back to the calling function and optionally returns a value. 'exit()' is a library function (not a keyword) that terminates the program.

Q65. ANSI C Keywords

The 'sizeof' keyword in C:

(A) Is a function that returns memory size

(B) ✓ Is an operator (keyword) that returns size in bytes

(C) Is only applicable to arrays

(D) Returns size in bits

Explanation: 'sizeof' is a compile-time unary operator (keyword) in C that returns the size of a data type or variable in bytes. It is not a function.

Q66. ANSI C Keywords

Which keyword is used to define a user-defined data type grouping different types?

(A) union

(B) ✓ struct

(C) enum

(D) typedef

Explanation: 'struct' (structure) groups variables of different data types under one name. 'union' also groups types but shares memory; 'enum' defines enumeration constants.

Q67. ANSI C Keywords

What does the 'auto' keyword mean in ANSI C?

(A) Automatic type deduction

(B) ✓ Variable with automatic storage duration (local variable)

(C) Automatic memory allocation

(D) Variable stored in heap

Explanation: 'auto' declares a variable with automatic storage duration — local to the block it is defined in. In C, local variables are 'auto' by default; explicitly writing 'auto' is redundant.

Q68. ANSI C Keywords

Which of the following is a VALID use of a C keyword?

(A) int while = 5;

(B) int for = 10;

(C) int count = while;

(D) ✓ int while_count = 5;

Explanation: 'while_count' is a valid identifier — 'while' is part of the name but the full identifier is not a keyword. Using 'while' alone as a variable name (A, B, C options) is illegal.

Q69. ANSI C Keywords

The keyword 'default' in C is used in:

(A) Function definitions

(B) ✓ Switch statements

(C) If-else statements

(D) Loop conditions

Explanation: 'default' is used in switch statements as the case that executes when no other case matches the expression value.

Q70. ANSI C Keywords

Which keyword declares a variable whose value persists between function calls?

(A) auto

(B) extern

(C) ✓ static

(D) register

Explanation: 'static' local variables retain their value between function calls. Their lifetime is the entire program, but their scope remains local to the function.

Q71. ANSI C Keywords

How many data type keywords does ANSI C89 include?

(A) 4

(B) ✓ 7

(C) 9

(D) 12

Explanation: ANSI C89 has 7 basic data type keywords: char, short, int, long, float, double, void (plus unsigned/signed modifiers and struct/union/enum compound types).

Q72. ANSI C Keywords

Which of the following keywords is related to enumeration in C?

(A) ✓ enum

(B) list

(C) set

(D) map

Explanation: 'enum' defines an enumerated type — a set of named integer constants. Example: enum Color {RED, GREEN, BLUE};

Q73. ANSI C Keywords

The keyword 'void' in C indicates:

(A) ✓ Empty or no value/type

(B) A null pointer

(C) An error condition

(D) Zero value

Explanation: 'void' indicates the absence of a type or value. Used for functions that return nothing (void function), or as void for a generic pointer.*

Q74. ANSI C Keywords

Which keyword transfers control to the next iteration of a loop, skipping the rest of the loop body?

(A) break

(B) goto

(C) ✓ continue

(D) next

Explanation: 'continue' skips the remaining statements in the current loop iteration and proceeds to the next iteration. 'break' exits the loop entirely.

Q75. ANSI C Keywords

Which of the following is NOT among the 32 ANSI C89 keywords?

(A) signed

(B) unsigned

(C) ✓ boolean

(D) volatile

Explanation: 'boolean' is not a C89 keyword. C89 has 'int' for boolean-like operations. '_Bool' was introduced in C99. 'signed', 'unsigned', and 'volatile' are all valid C89 keywords.

Q76. ANSI C Keywords

The 'case' keyword in C is used with:

(A) if-else statements

(B) ✓ switch statements

(C) for loops

(D) do-while loops

Explanation: 'case' is used within switch statements to define matching labels. It is always paired with 'switch' and 'break'.

Q77. ANSI C Keywords

Which pair of keywords is used for conditional compilation in C (via preprocessor)?

(A) if and else

(B) ✓ ifdef and ifndef

(C) switch and case

(D) for and while

Explanation: '#ifdef' and '#ifndef' are preprocessor directives (not technically C keywords but preprocessor keywords) for conditional compilation based on macro definitions.

Q78. ANSI C Keywords

What does 'unsigned' mean when used with a data type?

(A) The variable stores only negative values

(B) ✓ The variable can store only non-negative values (no sign bit)

(C) The variable cannot be modified

(D) The variable is uninitialized

Explanation: 'unsigned' modifier means the variable uses all bits for magnitude (no sign bit), allowing only non-negative values but with a larger positive range.

Q79. ANSI C Keywords

Which C89 keyword is used to exit a loop or switch statement?

(A) exit

(B) quit

(C) ✓ break

(D) stop

Explanation: 'break' terminates the execution of the nearest enclosing loop (for, while, do-while) or switch statement.

Q80. ANSI C Keywords

The keyword 'struct' in C is used to:

(A) Create functions

(B) Define arrays

(C) ✓ Define a composite data type

(D) Allocate memory dynamically

Explanation: 'struct' defines a structure — a composite data type that groups variables of different types. Example: struct Point {int x; int y;};

Q81. ANSI C Keywords

Which storage class keyword gives a variable file scope (visible only within the file)?

(A) extern

(B) register

(C) ✓ static (at file level)

(D) auto

Explanation: A 'static' variable at file (global) scope has internal linkage — it is accessible only within the file where it is defined, unlike regular global variables which have external linkage.

Q82. ANSI C Keywords

The 'do' keyword is used in C for:

(A) Defining functions

(B) ✓ A do-while loop that executes at least once

(C) Dynamic memory allocation

(D) Declaring constants

Explanation: 'do-while' loop guarantees at least one execution of the body before checking the condition (unlike 'while' which checks condition first).

Q83. ANSI C Keywords

Which of the following is a complete list of C loop keywords?

(A) for, while, do, loop

(B) ✓ for, while, do

(C) for, while, repeat

(D) for, while, do, until

Explanation: C has exactly three loop keywords: 'for', 'while', and 'do' (used in do-while). There is no 'loop', 'repeat', or 'until' keyword.

Q84. ANSI C Keywords

The keyword 'if' in C is used for:

(A) Input/output functions

(B) ✓ Conditional execution

(C) Infinite loops

(D) Integer format

Explanation: 'if' is a control flow keyword for conditional execution. The block executes only when the condition evaluates to non-zero (true).

Q85. ANSI C Keywords

Which C89 keyword is used in a switch statement to mark where to stop execution if no break is present?

(A) exit

(B) ✓ default

(C) end

(D) stop

Explanation: 'default' in a switch handles unmatched cases. Note: 'break' stops fall-through; 'default' is just the fallback case label, not a stop mechanism itself.

Section 4: Naming Conventions

Q86. Naming Conventions

Which naming convention uses underscores to separate words in an identifier?

(A) CamelCase

(B) PascalCase

(C) ✓ Snake_case

(D) Hungarian notation

Explanation: Snake_case uses underscores between words (e.g., student_name, total_count). It is commonly used for variable and function names in C.

Q87. Naming Conventions

In C programming, macros defined using #define are conventionally written in:

(A) lowercase only

(B) camelCase

(C) ✓ ALL_CAPS with underscores

(D) PascalCase

Explanation: By strong convention in C, macro constants (and function-like macros) are written in ALL_CAPS with underscores (e.g., MAX_SIZE, PI, BUFFER_LENGTH).

Q88. Naming Conventions

The naming convention where each word starts with a capital letter (e.g., StudentName) is called:

(A) Snake case

(B) Camel case

(C) ✓ Pascal case

(D) Hungarian notation

Explanation: PascalCase (also called UpperCamelCase) capitalizes the first letter of every word. It is commonly used for struct names and type names in C.

Q89. Naming Conventions

The naming convention where the first word is lowercase and subsequent words start with uppercase (e.g., studentName) is:

(A) Snake case

(B) ✓ lowerCamelCase (camelCase)

(C) Pascal case

(D) Hungarian notation

Explanation: lowerCamelCase (camelCase) starts with a lowercase letter for the first word, then capitalizes subsequent words. Common in Java and often used in C function/variable names.

Q90. Naming Conventions

What is 'Hungarian notation' in C programming?

(A) Writing identifiers in Hungarian language

(B) ✓ Prefixing identifiers with type information

(C) Using underscores for all identifiers

(D) Writing all identifiers in uppercase

Explanation: Hungarian notation prefixes variable names with a code indicating their type (e.g., iCount for integer count, szName for zero-terminated string, bFlag for boolean).

Q91. Naming Conventions

According to common C conventions, a global variable is often distinguished by:

(A) Using all lowercase

(B) ✓ Adding 'g_' prefix or using longer descriptive names

(C) Starting with a digit

(D) Using only single character names

Explanation: A common C convention prefixes global variables with 'g_' (e.g., g_totalCount) to distinguish them from local variables and make their global scope explicit in the code.

Q92. Naming Conventions

Which is the BEST name for a variable storing a student's age according to good naming conventions?

(A) a

(B) age

(C) studentAge

(D) ✓ Both B and C

Explanation: Both 'age' and 'studentAge' are good names — meaningful and descriptive. 'a' is too cryptic. In a student-related context, 'age' is clear; 'studentAge' adds more context.

Q93. Naming Conventions

Which statement about C naming conventions is INCORRECT?

(A) Constants defined with #define are traditionally in UPPER_CASE

(B) Structure type names often use PascalCase

(C) Local variables use camelCase or snake_case

(D) ✓ All C identifiers must use Hungarian notation

Explanation: Hungarian notation is a convention, not a requirement. C does not mandate any naming style. The others represent common but not mandatory conventions.

Q94. Naming Conventions

Which identifier follows good naming practice for a constant maximum buffer size?

(A) mbs

(B) maxBufferSize

(C) ✓ MAX_BUFFER_SIZE

(D) max

Explanation: 'MAX_BUFFER_SIZE' follows the C convention of ALL_CAPS with underscores for constants. It is descriptive and clearly indicates it's a constant maximum buffer size.

Q95. Naming Conventions

For struct type names in C, which convention is commonly recommended?

(A) ALL_CAPS

(B) snake_case

(C) ✓ PascalCase or tag_t suffix

(D) Hungarian prefix

Explanation: Struct type names commonly use PascalCase (e.g., StudentRecord) or a _t suffix convention (e.g., student_t). This distinguishes type names from variable names.

Q96. Naming Conventions

A function that calculates the area of a circle should ideally be named:

(A) f1()

(B) x()

(C) ✓ calculateCircleArea()

(D) calc()

Explanation: 'calculateCircleArea()' is descriptive and clearly expresses the function's purpose. Short cryptic names like 'f1', 'x', or 'calc' reduce code readability and maintainability.

Q97. Naming Conventions

Which of the following follows the snake_case naming convention?

(A) studentName

(B) StudentName

(C) ✓ student_name

(D) STUDENTNAME

Explanation: 'student_name' uses underscores between words in lowercase — this is snake_case. 'studentName' is camelCase; 'StudentName' is PascalCase; 'STUDENTNAME' is UPPER_CASE.

Q98. Naming Conventions

In C, using a single-letter variable name like 'i' is acceptable in which situation?

(A) Never — all variables must have descriptive names

(B) ✓ For loop counter variables (short scope)

(C) For all global variables

(D) Only for character variables

Explanation: Single-letter variables like 'i', 'j', 'k' are widely accepted as loop counters because their scope is minimal and their purpose is clear. For other uses, descriptive names are preferred.

Q99. Naming Conventions

The prefix 'is' in identifiers (e.g., isValid, isEmpty) conventionally indicates:

(A) Integer storage

(B) ✓ A boolean/flag variable or function returning true/false

(C) Internal scope

(D) An integer size type

Explanation: The 'is' prefix is a common convention to indicate boolean semantics — the variable or function represents or returns a true/false value (e.g., isValid, isEmpty, isReady).

Q100. Naming Conventions

Which of the following identifier names violates standard C naming conventions most severely?

(A) total_cost

(B) TotalCost

(C) TOTAL_COST

(D) ✓ tc

Explanation: While all are valid identifiers, 'tc' is a severe violation of good naming conventions — it is cryptic and fails to convey meaning. The others are readable and self-documenting.

Section 5: Reserved Words

Q101. Reserved Words

Reserved words in C refer to:

(A) Words reserved for future expansion only

(B) ✓ Keywords plus implementation-reserved identifiers

(C) Only the 32 standard keywords

(D) Variable names that cannot be changed

Explanation: Reserved words include both the 32 standard keywords AND identifiers reserved by the implementation (those beginning with __ or _ followed by uppercase). Together they form the complete set of reserved names.

Q102. Reserved Words

Which of the following is a keyword added in C99 (not in ANSI C89)?

(A) const

(B) volatile

(C) ✓ inline

(D) extern

Explanation: 'inline' was introduced in C99 as a keyword (though many compilers supported it as an extension before C99). 'const', 'volatile', and 'extern' are all ANSI C89 keywords.

Q103. Reserved Words

Why can't reserved words be used as identifiers in C?

(A) They are too long for the compiler to handle

(B) ✓ They have predefined meanings and the compiler uses them for specific syntactic purposes

(C) They cause memory overflow

(D) They are case-sensitive

Explanation: Reserved words/keywords have fixed, predefined meanings to the C compiler. Using them as identifiers would create ambiguity in parsing the source code.

Q104. Reserved Words

Which of the following correctly lists only ANSI C89 keywords?

(A) ✓ auto, break, case, char, const, continue, default, do

(B) auto, break, inline, restrict, _Bool

(C) inline, restrict, _Bool, _Complex

(D) for, while, loop, repeat

Explanation: All of auto, break, case, char, const, continue, default, do are genuine ANSI C89 keywords. 'inline', 'restrict', '_Bool', '_Complex' were added in C99. 'loop' and 'repeat' are not C keywords at all.

Q105. Reserved Words

Which of the following is a complete group of C89 control flow keywords?

(A) ✓ if, else, for, while, do, switch, case, break, continue, goto, return, default

(B) if, else, for, while, do, switch, case, break, continue, goto, return, default, yield

(C) if, for, while, switch

(D) if, else, for, while

Explanation: The complete set of C89 control flow keywords is: if, else, for, while, do, switch, case, break, continue, goto, return, default. 'yield' does not exist in C.

Q106. Reserved Words

The word 'NULL' in C is:

(A) A keyword

(B) ✓ A reserved word but not a keyword — it is a macro

(C) An invalid identifier

(D) A standard data type

Explanation: 'NULL' is a preprocessor macro (defined in several headers) that expands to a null pointer constant. It is NOT a keyword, though it is effectively reserved by convention and should not be redefined.

Q107. Reserved Words

Which group contains ONLY storage class keywords in C89?

(A) ✓ auto, register, static, extern

(B) auto, register, static, extern, inline

(C) static, extern, const, volatile

(D) auto, register, const

Explanation: The four storage class specifiers in C89 are: auto, register, static, extern. 'inline' is C99; 'const' and 'volatile' are type qualifiers, not storage classes.

Q108. Reserved Words

'_Bool', '_Complex', and '_Imaginary' are keywords in:

(A) ANSI C89/C90

(B) ✓ C99 and later

(C) C11 only

(D) GNU C extension only

Explanation: '_Bool', '_Complex', and '_Imaginary' were introduced as keywords in C99. They are not present in ANSI C89.

Q109. Reserved Words

Which of the following is NOT a type qualifier keyword in ANSI C89?

(A) const

(B) volatile

(C) ✓ restrict

(D) Both A and B

Explanation: 'restrict' is a type qualifier introduced in C99. In C89, only 'const' and 'volatile' are type qualifiers. 'restrict' tells the compiler that a pointer is the only way to access the pointed-to object.

Q110. Reserved Words

Which ANSI C keyword is used to define a group of related integer constants?

(A) struct

(B) union

(C) ✓ enum

(D) typedef

Explanation: 'enum' defines an enumeration: a list of named integer constants. Example: enum Day {MON=1, TUE, WED, THU, FRI, SAT, SUN};

Q111. Reserved Words

The keyword 'union' in C is similar to 'struct' except:

(A) It can only hold integer types

(B) ✓ All members share the same memory location

(C) It is only for global variables

(D) It uses more memory than struct

Explanation: In a 'union', all members share the same memory location. The size of the union is the size of its largest member. Only one member can store a value at a time.

Q112. Reserved Words

Which of the following is the correct assertion about C keywords?

(A) Keywords can be redefined using typedef

(B) ✓ Keywords can be used as structure member names

(C) Keywords cannot appear as any part of a valid identifier

(D) Keywords can be used as variable names if capitalized

Explanation: Structure member names (fields) have their own namespace and CAN use keyword names (e.g., struct S { int for; }; is technically valid, though extremely poor practice and compiler-dependent).

Q113. Reserved Words

How many type qualifier keywords exist in ANSI C89?

(A) 1

(B) ✓ 2

(C) 3

(D) 4

Explanation: ANSI C89 has exactly 2 type qualifiers: 'const' (read-only) and 'volatile' (value may change externally). C99 added 'restrict' as a third qualifier.

Q114. Reserved Words

The 'signed' keyword in C:

(A) Is illegal in C89

(B) ✓ Explicitly declares a type that can hold negative values

(C) Is only for floating-point types

(D) Makes a variable immutable

Explanation: 'signed' explicitly states that an integer type can hold negative values (uses sign bit). For 'int', 'signed' is the default, but it can be explicitly specified: signed int, signed char.

Q115. Reserved Words

Which of the following is NOT a reserved word in the context of ANSI C?

(A) static

(B) ✓ string

(C) struct

(D) sizeof

Explanation: 'string' is not a keyword or reserved word in C. In C, strings are represented as character arrays (char[]). 'static', 'struct', and 'sizeof' are all ANSI C keywords.

Q116. Reserved Words

A C compiler can use identifiers starting with '__' for its internal purposes because:

(A) Double underscore is a debug convention

(B) ✓ Such identifiers are reserved by the C standard for the implementation

(C) Compilers define this in their documentation

(D) It is just a tradition with no standard basis

Explanation: The C standard (Section 7.1.3) reserves identifiers beginning with double underscore (__) for the implementation. This is in the standard to prevent name collisions.

Q117. Reserved Words

Which C89 keyword category includes: short, long, signed, unsigned?

(A) Storage class specifiers

(B) Type qualifiers

(C) ✓ Type modifiers

(D) Control flow keywords

Explanation: 'short', 'long', 'signed', 'unsigned' are type modifiers (or type specifiers) that modify the range and representation of the basic data types.

Q118. Reserved Words

The keyword 'char' in C defines a type that stores:

(A) Only lowercase letters

(B) ✓ Integer values typically representing a character (1 byte)

(C) Unicode characters

(D) Boolean values

Explanation: 'char' stores a single character as a 1-byte integer. Its value can be used as an ASCII code. Whether 'char' is signed or unsigned is implementation-defined.

Q119. Reserved Words

Which is the correct classification of the word 'printf' in C?

(A) Keyword

(B) Reserved word

(C) ✓ Standard library function identifier

(D) Preprocessor directive

Explanation: 'printf' is a standard library function declared in . It is NOT a keyword. It is an identifier that the standard library defines, and technically could be redefined (though doing so is extremely poor practice).

Q120. Reserved Words

The complete set of ANSI C89 data type keywords includes:

(A) char, int, float, double, void

(B) char, short, int, long, float, double, void

(C) ✓ char, short, int, long, float, double, void, signed, unsigned

(D) char, int, long, float, double, string, void

Explanation: The complete C89 type keywords are: char, short, int, long, float, double, void (basic types) plus signed and unsigned (modifiers). Together they cover all standard type specifications.

MOCK TEST

30 Questions | Suggested Time: 45 Minutes | 1 Mark Each

INSTRUCTIONS: This mock test contains 30 multiple-choice questions. Each question carries 1 mark. There is NO negative marking. Read each question carefully before answering. Answer all questions. Answers are provided separately in the Answer Key section.

Q1. MOCK TEST

Which of the following is a valid ANSI C identifier?

(A) 2ndValue
(B) _2ndValue
(C) 2_ndValue
(D) nd-2Value

Q2. MOCK TEST

How many keywords are defined in ANSI C89/C90?

(A) 27
(B) 32
(C) 35
(D) 40

Q3. MOCK TEST

Which of the following is NOT a C89 keyword?

(A) volatile
(B) const
(C) inline
(D) extern

Q4. MOCK TEST

An identifier in C can start with which of the following?

(A) A digit
(B) A special character
(C) A letter or underscore
(D) Any character

Q5. MOCK TEST

C identifiers are:

(A) Case-insensitive
(B) Case-sensitive
(C) Converted to uppercase internally
(D) Limited to 8 characters

Q6. MOCK TEST

Which storage class retains value between function calls?

(A) auto
(B) extern
(C) register
(D) static

Q7. MOCK TEST

The keyword 'sizeof' in C is:

(A) A function
(B) A macro
(C) A compile-time operator
(D) A type modifier

Q8. MOCK TEST

Which of the following is a valid identifier?

(A) float
(B) Float
(C) FLOAT!
(D) 3float

Q9. MOCK TEST

What is the maximum significant length for internal identifiers in C89?

(A) 8
(B) 16
(C) 31
(D) 64

Q10. MOCK TEST

Which keyword prevents compiler optimization of a variable?

(A) const
(B) register
(C) volatile
(D) static

Q11. MOCK TEST

Macro names in C are conventionally written in:

(A) camelCase
(B) snake_case
(C) ALL_CAPS
(D) PascalCase

Q12. MOCK TEST

Which of the following is NOT allowed in a C identifier?

(A) Underscore
(B) Digit
(C) Uppercase letter
(D) Hyphen

Q13. MOCK TEST

The 'extern' keyword is used to:

(A) Create external variables
(B) Declare a variable defined elsewhere
(C) Store variables externally
(D) Access OS functions

Q14. MOCK TEST

Which keyword is used to exit from the innermost loop?

(A) exit
(B) continue
(C) break
(D) return

Q15. MOCK TEST

The 'register' keyword is a hint to the compiler to store the variable in:

(A) Heap memory (B) Stack memory

(C) CPU register (D) Hard disk

Q16. MOCK TEST

Which of the following is a C89 type qualifier?

(A) inline (B) restrict

(C) volatile (D) _Bool

Q17. MOCK TEST

Identifiers beginning with '__' (double underscore) are:

(A) Required for global variables (B) Reserved for implementation use

(C) Used for array names (D) Recommended for constants

Q18. MOCK TEST

Which of the following is a valid C identifier?

(A) int (B) _int

(C) Int (D) Both B and C

Q19. MOCK TEST

'continue' in a C loop:

(A) Exits the loop (B) Restarts the program

(C) Skips to the next iteration (D) Pauses execution

Q20. MOCK TEST

Which of the following naming styles uses underscores between words?

(A) camelCase (B) PascalCase

(C) snake_case (D) Hungarian

Q21. MOCK TEST

The keyword 'typedef' is used to:

(A) Define a new data type from scratch (B) Create an alias for an existing type

(C) Declare a type-safe variable (D) Convert types automatically

Q22. MOCK TEST

In 'enum Color {RED, GREEN, BLUE};', what is RED's default value?

(A) -1 (B) 0

(C) 1 (D) Undefined

Q23. MOCK TEST

Which identifier is INVALID in C?

(A) _ (B) __

(C) a1b2c3 (D) return

Q24. MOCK TEST

The 'auto' keyword in C89 is:

(A) Used for type deduction (B) Redundant for local variables (default storage class)

(C) Required for all local variables (D) Used for automatic memory allocation

Q25. MOCK TEST

Which of the following is TRUE?

(A) 'main' is a keyword in C (B) 'printf' is a keyword in C

(C) 'NULL' is a keyword in C (D) 'while' is a keyword in C

Q26. MOCK TEST

How many type qualifiers does ANSI C89 have?

(A) 1 (B) 2

(C) 3 (D) 4

Q27. MOCK TEST

For external identifiers, ANSI C89 guarantees significance up to how many characters?

(A) 6 (B) 8

(C) 16 (D) 31

Q28. MOCK TEST

Which C keyword defines a user-defined type that shares memory among members?

(A) struct (B) union

(C) enum (D) typedef

Q29. MOCK TEST

Which is NOT a loop keyword in C?

(A) for (B) while

(C) do (D) repeat

Q30. MOCK TEST

The identifier 'NULL' in C is best described as:

(A) A keyword (B) A data type

(C) A preprocessor macro (D) A reserved keyword

COMPLETE ANSWER KEY

All Sections (Q1–Q120) + Mock Test (Q1–Q30)

Section 1: Rules for Identifiers

Q1: (C)	Q2: (B)	Q3: (C)	Q4: (B)	Q5: (D)	Q6: (C)	Q7: (B)	Q8: (A)	Q9: (C)	Q10: (C)
Q11: (B)	Q12: (B)	Q13: (B)	Q14: (B)	Q15: (C)	Q16: (B)	Q17: (C)	Q18: (B)	Q19: (B)	Q20: (B)
Q21: (B)	Q22: (B)	Q23: (B)	Q24: (B)	Q25: (B)					

Section 2: Valid and Invalid Identifiers

Q26: (B)	Q27: (D)	Q28: (C)	Q29: (D)	Q30: (C)	Q31: (D)	Q32: (C)	Q33: (D)	Q34: (B)	Q35: (C)
Q36: (D)	Q37: (D)	Q38: (D)	Q39: (C)	Q40: (B)	Q41: (B)	Q42: (B)	Q43: (B)	Q44: (D)	Q45: (B)
Q46: (C)	Q47: (B)	Q48: (B)	Q49: (B)	Q50: (C)	Q51: (B)	Q52: (B)	Q53: (B)	Q54: (D)	Q55: (B)

Section 3: ANSI C Keywords

Q56: (C)	Q57: (C)	Q58: (B)	Q59: (B)	Q60: (B)	Q61: (B)	Q62: (C)	Q63: (B)	Q64: (B)	Q65: (B)
Q66: (B)	Q67: (B)	Q68: (D)	Q69: (B)	Q70: (C)	Q71: (B)	Q72: (A)	Q73: (A)	Q74: (C)	Q75: (C)
Q76: (B)	Q77: (B)	Q78: (B)	Q79: (C)	Q80: (C)	Q81: (C)	Q82: (B)	Q83: (B)	Q84: (B)	Q85: (B)

Section 4: Naming Conventions

Q86: (C)	Q87: (C)	Q88: (C)	Q89: (B)	Q90: (B)	Q91: (B)	Q92: (D)	Q93: (D)	Q94: (C)	Q95: (C)
Q96: (C)	Q97: (C)	Q98: (B)	Q99: (B)	Q100: (D)					

Section 5: Reserved Words

Q101: (B)	Q102: (C)	Q103: (B)	Q104: (A)	Q105: (A)	Q106: (B)	Q107: (A)	Q108: (B)	Q109: (C)	Q110: (C)
Q111: (B)	Q112: (B)	Q113: (B)	Q114: (B)	Q115: (B)	Q116: (B)	Q117: (C)	Q118: (B)	Q119: (C)	Q120: (C)

Mock Test Answer Key

Q1: (B)	Q2: (B)	Q3: (C)	Q4: (C)	Q5: (B)	Q6: (D)	Q7: (C)	Q8: (B)	Q9: (C)	Q10: (C)
Q11: (C)	Q12: (D)	Q13: (B)	Q14: (C)	Q15: (C)	Q16: (C)	Q17: (B)	Q18: (D)	Q19: (C)	Q20: (C)

Q21: (B)	Q22: (B)	Q23: (D)	Q24: (B)	Q25: (D)	Q26: (B)	Q27: (A)	Q28: (B)	Q29: (D)	Q30: (C)

Mock Test Score Interpretation

Score Range	Interpretation	Recommendation
27–30 (90%+)	Excellent	You are exam-ready! Revise weak areas.
22–26 (73%–87%)	Good	Practice more questions. Focus on grey areas.
16–21 (53%–70%)	Average	Re-read topic notes. Attempt more mock tests.
Below 16 (<53%)	Needs Improvement	Study the concept sections thoroughly.

© 2025 OAVS Exam Series | This material is prepared exclusively for OAVS Computer Teacher (TGT) examination preparation. Based on ANSI C (C89/C90) standard.